Materia: Sistemas Operativos II

1. Actividad S2-A2

Gestión de procesos en Linux Ubuntu desde la Terminal

Objetivo general

Al finalizar esta actividad, serás capaz de trabajar de manera eficaz con los procesos en la terminal de Linux Ubuntu. Esto incluye monitorear, controlar y comprender el impacto de estos procesos en el rendimiento general del sistema.

Objetivos específicos

- Comprender el concepto de proceso en sistemas operativos Linux.
- Familiarizarse con los comandos esenciales para la gestión de procesos en Ubuntu.
- Aprender a monitorear el estado y los recursos consumidos por los procesos.
- Adquirir la habilidad de iniciar, detener, pausar y reanudar procesos.
- Entender la importancia de la gestión de procesos para la estabilidad y eficiencia del sistema.

Conceptos previos

Es fundamental conocer el significado de la información que brindarán los comandos a utilizar:

- Proceso: Es una instancia de un programa en ejecución. Cada vez que abres una aplicación, el sistema operativo crea un proceso (o varios) para ejecutarla.
- PID (Process ID): Es un número único que el sistema operativo asigna a cada proceso. Es como la cédula de identidad de un proceso.
- PPID (Parent Process ID): Es el PID del proceso que inició el proceso actual. Los procesos tienen una jerarquía, como un árbol genealógico.

- Usuario (User): Es el usuario propietario del proceso. Los procesos pueden ser ejecutados por el usuario actual, por el usuario root (administrador) o por otros usuarios del sistema.
- CPU Usage (%CPU): Indica el porcentaje de tiempo de CPU que un proceso está utilizando.
- Memory Usage (%MEM): Indica el porcentaje de memoria RAM que un proceso está consumiendo.
- Estado del Proceso (STAT/S): Representa el estado actual de un proceso (ejecutándose, en espera, detenido, zombie, etc.).
- Prioridad (PRI/NI): Determina la importancia de un proceso en la cola de ejecución de la CPU. Una prioridad más baja significa que el proceso tiene más posibilidades de ser ejecutado. NI (Nice value) es un valor que el usuario puede ajustar para influir en la prioridad.

1.2 Software necesario

- Máquina virtual con el sistema operativo Linux Ubuntu Desktop o Server.

  Aclaración: Si su instalación no cuenta con interfaz gráfica de usuario (GUI), como por ejemplo Linux Ubuntu Server, vea el apartado **Terminales virtuales** al final de este documento para saber cómo trabajar con más de una terminal al mismo tiempo.

1.3 Desarrollo de la actividad

1.3.1 Visualización y monitoreo de procesos

En este punto, aprenderemos a ver qué procesos están corriendo en nuestro sistema.

a. Comando ps (process status)

Se utiliza para enumerar los procesos que se están ejecutando actualmente.

- **Función:** Muestra información sobre los procesos que se están ejecutando.
- **Ejemplos:**
  - ps: Muestra los procesos asociados a la terminal actual.

- ps -e: Muestra todos los procesos que se están ejecutando en el sistema. (-e significa "every process").
- ps -f: Muestra un formato de listado completo, con más detalles (usuario, PID, PPID, etc.). (-f significa "full format").
- ps aux: Es una de las combinaciones más comunes.
    - a: Muestra procesos de todos los usuarios.
    - u: Muestra procesos asociados a un usuario específico, con formato de usuario/proceso.
    - x: Muestra procesos sin terminal de control.
    - Esta combinación te dará una lista completa y detallada de todos los procesos en el sistema.

- **Práctica:**
    - Abre una terminal y ejecuta **ps**. ¿Qué procesos vez?
    - Ejecuta **ps -e**. ¿Cuántos procesos diferentes aparecen? Compara con el resultado anterior.
    - Ejecuta **ps aux**. Intenta identificar procesos que conozcas (por ejemplo, tu navegador, el terminal que estás usando). ¿Qué información útil te proporciona esta salida?
    - Usa **ps aux | grep <nombre_proceso>** para buscar un proceso específico (ej. ps aux | grep firefox). Si Firefox no está abierto, ábrelo y vuelve a intentarlo.

b. Comando: top (Table Of Processes)

El comando **top** proporciona una vista dinámica y en tiempo real de los procesos. Es ideal para monitorear el rendimiento del sistema.

- **Función:** Muestra un resumen del sistema y una lista de los procesos que más recursos están consumiendo, actualizándose periódicamente.
- **Ejemplo:**
    - top: Inicia la interfaz interactiva de top.
- **Práctica:**
    - Ejecuta top. Observa la información en la parte superior (uptime, carga promedio, tareas, uso de CPU, uso de memoria).
    - Identifica los procesos que más CPU o memoria están consumiendo.
    - Mientras top está corriendo, abre otra aplicación (ej. un navegador, un editor de texto) y observa cómo cambian los valores en top.
    - Presiona “q” para salir de top.

- Dentro de top, presiona **M** para ordenar los procesos por uso de memoria y **P** para ordenar por uso de CPU.

1.3.2. Control de Procesos

Ahora que sabemos cómo ver los procesos, aprenderemos a interactuar con ellos.

a. Comando: kill (Terminate a process)

El comando kill se usa para enviar señales a los procesos, siendo la señal por defecto SIGTERM (terminar amistosamente).

- **Función:** Envía una señal a un proceso especificado por su *PID* para controlarlo (generalmente para terminarlo).
- **Ejemplos:**
  - kill <PID>: Envía la señal SIGTERM (15) al proceso, pidiéndo que finalice de forma ordenada.
  - kill -9 <PID>: Envía la señal SIGKILL (9), que fuerza la terminación inmediata del proceso sin permitirle guardar cambios o limpiar. Úsalo con precaución, sólo si kill <PID> no funciona.
- **Práctica:**
  - Abre una aplicación simple que no sea crítica para el sistema (por ejemplo, nano o gedit si lo tienes instalado; si no, puedes instalarlo con *sudo apt install gedit*).
  - Usa **ps aux | grep nano** (o grep gedit) para encontrar su PID.
  - Ejecuta **kill <PID_de_nano/gedit>**. ¿Qué sucede con la aplicación?
  - Si la aplicación no cierra, intenta forzar el cierre con **kill -9 <PID_de_nano/gedit>.**
  - **PRECAUCIÓN:** ¡No intentes terminar procesos importantes del sistema como init, systemd, o tu propio terminal! Esto podría bloquear tu sistema.

1.3.3. Modos de Ejecución

Todo sistema operativo moderno tiene la capacidad de ejecutar múltiples procesos de manera eficiente y controlada. Estos procesos pueden ser desde aplicaciones *interactivas* (con interacción con el usuario) hasta tareas automatizadas que operan sin intervención del usuario.
Para gestionar esta diversidad de actividades, los sistemas operativos distinguen entre procesos que se ejecutan en **primer plano (foreground)** y aquellos que lo hacen en **segundo plano (background)**.

Esta distinción no solo afecta la forma en que el usuario interactúa con el sistema, sino también cómo se asignan los recursos y cómo se priorizan las tareas.

Características de ejecutar un proceso en primer plano (foreground)

- Hay una interacción directa entre el programa y el usuario
- Bloquea la terminal para otros usos mientras el programa se está ejecutando, debemos esperar a que finalice para continuar con la próxima tarea.
- Normalmente los sistemas operativos le asignan una mayor prioridad, especialmente cuando se trata de sistemas compartidos (time sharing) que priorizan el tiempo de respuesta al usuario.

Características de ejecutar un proceso en segundo plano (background)

- No requiere interacción directa con el usuario.
- No bloquea la terminal y queda libre para usar en otras tareas
- El sistema le asignará recursos al proceso de acuerdo a las políticas definidas.
- Es ideal para tareas largas y automatizadas

Observación

No todas las tareas pueden ser ejecutadas en background, aquellas que tienen una interacción necesaria con el usuario no se pueden ejecutar en este modo.

a. Comando: fg (Foreground) y bg (Background)

Estos comandos se utilizan para gestionar trabajos (procesos) que se ejecutan en primer o segundo plano en una terminal.

- **Función:**
  - **fg**: Trae un trabajo que está en segundo plano al primer plano (interactivo).
  - **bg**: Envía un trabajo que está detenido al segundo plano para que continúe ejecutándose.
- **Ejemplos:**
  - **firefox &**: Inicia Firefox en segundo plano. El **"&"** al final del comando envía el proceso al background inmediatamente.
  - **sleep 1000**: Inicia un proceso que espera 1000 segundos. Si lo ejecutas y luego presionas Ctrl+Z, el proceso se detendrá (pausará).
  - **jobs:** Muestra los trabajos (procesos) que están en segundo plano o detenidos en la terminal actual.
  - **bg %1**: Envía el trabajo número 1 (según jobs) al segundo plano.

- **fg %1**: Trae el trabajo número 1 al primer plano.

- **Práctica:**
  - Ejecuta **sleep 1000**. Inmediatamente después de ejecutarlo, presiona Ctrl+Z. ¿Qué sucede? (El proceso se detendrá).
  - Ejecuta **jobs**. Deberías ver sleep listado como "Stopped".
  - Ejecuta **bg %1** (asumiendo que sleep es el trabajo número 1). El proceso sleep debería continuar ejecutándose en segundo plano. Observa que la terminal vuelve a estar disponible.
  - Ejecuta **fg %1**. El proceso sleep vuelve al primer plano. Ahora la terminal está "bloqueada" por sleep nuevamente.
  - Presiona **Ctrl+C** para terminar sleep ahora que está en primer plano.
  - Intenta iniciar un programa como **gedit &** o **firefox &**. Observa cómo la terminal permanece disponible después de iniciar el programa.

1.3.4 Prioridad de Procesos

Entender cómo el sistema operativo asigna recursos a los procesos es crucial. La prioridad es uno de esos mecanismos.

a. Comando: nice y renice

Estos comandos permiten ajustar la "niceness" (amabilidad) de un proceso, lo que afecta su prioridad de ejecución. Un valor "nice" alto significa que el proceso es más "amable" y le dará más oportunidades a otros procesos de ejecutarse, es decir, tiene menor prioridad.

- **Función:**
  - **nice**: Inicia un comando con una prioridad de ejecución específica (valor "nice").
  - **renice**: Cambia la prioridad de un proceso que ya está en ejecución.
- **Valores de "Nice":** Van de -20 (la prioridad más alta, menos amable) a 19 (la prioridad más baja, más amable). El valor por defecto es 0.

- **Ejemplos:**
  - **nice -n 10 <comando>**: Inicia <comando> con un valor "nice" de 10 (menor prioridad).
  - **renice +5 -p <PID>**: Cambia la prioridad del proceso con <PID> a 5 (menor prioridad).
  - **sudo renice -10 -p <PID>**: Para aumentar la prioridad (valores negativos), se necesitan permisos de superusuario (sudo).

- **Práctica:**
    - Abre dos terminales.
    - En la primera terminal, ejecuta un comando que consuma CPU, como **yes > /dev/null**. (Este comando genera un flujo constante de 'y's).
    - En la segunda terminal, usa **ps aux | grep yes** para encontrar el PID de ese proceso.
    - En la segunda terminal, ejecuta **top** o **htop** y observa el uso de CPU del proceso yes. Anota su valor NI.
    - Ahora, intenta cambiar su prioridad: **renice +10 -p <PID_de_yes>**.
    - Vuelve a **top** o **htop** y observa cómo ha cambiado su NI y si su uso de CPU se ha reducido (aunque en un sistema con poca carga, esto puede ser difícil de apreciar).
    - Intenta aumentar su prioridad (requiere sudo): **sudo renice -5 -p <PID_de_yes>**. Observa el cambio.
    - Finalmente, presiona **Ctrl+C** en la primera terminal para detener el comando yes.